

Android Activity Lab: Lifecycle & Communication Between Activities

Lab Worksheet

Instructions

Starting from a two-activity demo project (`MainActivity` launching `SecondActivity` and receiving a result back), complete the modifications below in order: **Easy** → **Medium** → **Challenge**. Every task describes a symptom, not the exact broken line — you'll need to run the app, read the code, and work out which callback or which piece of Intent-passing is actually at fault before you fix it. Record your answers to the “Verify” and “Explain” prompts in the space provided — these will be checked at the end of the lab.

Easy Tasks

Task 1: Diagnose Data That Never Arrives

`MainActivity` sends a value to `SecondActivity` using `intent.putExtra(...)`, and `SecondActivity` reads it back with `intent.getStringExtra(...)`. The screen opens without crashing, but the value it should display always falls back to a default placeholder instead of what was actually sent.

Your job: Compare the exact key string used in `putExtra()` against the one used in `getStringExtra()`. Fix whichever one doesn't match — ideally by replacing both with a single shared constant so they can never drift apart again.

Verify: Enter a value in `MainActivity`, open `SecondActivity` — it should display exactly what you entered, not the placeholder.

Explain: The app didn't crash even though the key didn't match — what does `getStringExtra()` return when the key it's asked for isn't found in the Intent?

Task 2: Diagnose an Activity That Refuses to Open

Tapping the button that should open `SecondActivity` instead crashes the app immediately with an `ActivityNotFoundException`.

Your job: `SecondActivity`'s Kotlin file compiles fine and the Intent code looks correct — so look somewhere Android configuration lives outside your Kotlin code entirely, and fix what's missing there.

Verify: Tap the button — `SecondActivity` should open normally, with no crash.

Explain: Why does Android require every activity to be declared in one particular file before it can ever be launched, even though the class itself compiles without any errors?

Task 3: Diagnose a Result That's Always Reported as Cancelled

`SecondActivity` correctly attaches the picked value to its result Intent before finishing. But no matter what the user picks, `MainActivity`'s callback always runs the “cancelled” branch instead of reading the picked value.

Your job: The data is present — something else about how the result is being finished is wrong. Find it and fix it.

Verify: Pick a value in `SecondActivity` — `MainActivity` should show it, not a “cancelled” message.

Explain: An `ActivityResult` carries two separate pieces of information back to the caller. Name both of them, and explain why having correct data attached isn't enough on its own.

Medium Tasks

Task 4: Diagnose a Returned Value That Disappears on Rotation

Pick a value in `SecondActivity`, return to `MainActivity`, and confirm it displays correctly. Now rotate the device without doing anything else — the displayed value resets to its default, even though you never launched `SecondActivity` again.

Your job: The activity-result callback only fires once, right after the round trip — it won't fire again just because the screen rotated. Persist the received value some other way so it survives rotation on its own.

Verify: Complete a pick, confirm the value shows, then rotate the device — it should still be showing, not reset.

Explain: Why doesn't rotating the device re-trigger the activity-result callback the way it might seem like it should?

Task 5: Diagnose a Launcher That Crashes on Tap

The button that's supposed to open `SecondActivity` crashes the instant it's tapped, with an error about registering for activity results too late.

Your job: Somewhere, the call that registers the result launcher has ended up inside a method body instead of being declared as a class-level property. Move it back to where it needs to live, and be ready to explain why the location matters.

Verify: Tap the button — `SecondActivity` should open and return results with no crash.

Explain: Which lifecycle callback must not have already started running before a result launcher is registered, and why does Kotlin property initialization order make top-of-class declaration the only safe place for it?

Task 6: Diagnose a False “Success” on Cancel

Open `SecondActivity` and immediately press the system Back button without picking anything. `MainActivity` still reports a successful pick with a default or stale value, instead of correctly reporting that nothing was chosen.

Your job: Find where `SecondActivity` is calling `setResult(RESULT_OK, ...)` and work out why it's being called before the user has actually made a choice. Move it to the point where a real selection happens.

Verify: Open `SecondActivity` and press Back immediately — `MainActivity` should report a cancellation, not a false success.

Explain: Which callback is guaranteed to run when the user presses Back on `SecondActivity`, and why does calling `setResult()` too early undermine the cancel path even though `finish()` still runs correctly either way?

Challenge Tasks

Task 7: Diagnose a Stack of Duplicate Activities

A button meant to jump to a specific screen (e.g. from a repeated action, like tapping a “details” shortcut more than once) launches a brand-new instance of the same target Activity every single time it’s tapped, stacking several identical copies on the back stack — the user has to press Back repeatedly just to leave.

Your job: Change the target activity’s launch mode so repeated launches reuse the existing instance instead of creating new ones, and add the callback that lets a reused instance handle the newly arriving Intent.

Verify: Trigger the launch several times in a row — only one instance of the activity should ever exist, and pressing Back once should return directly to the previous screen.

Explain: What’s the difference between `onCreate()` and the callback you added, in terms of when each one runs — and why is relying on `onCreate()` alone not enough once the launch mode changes?

Task 8: Diagnose a “Waiting” Indicator That Gets Stuck

MainActivity shows a “Waiting for result...” message the moment it launches SecondActivity, meant to clear once a result comes back — and normally it does. But with Developer Options → “**Don’t keep activities**” enabled, if the system kills MainActivity in the background while SecondActivity is still open, the returned value *does* still come back correctly (the modern Activity Result API is built to survive this) — yet the “Waiting...” message stays on screen forever, out of sync with the data that arrived.

Your job: Work out why the returned *data* survives process death but the “waiting” *flag* doesn’t, and fix the flag the same way you’d fix any other piece of state lost to process death.

Verify: With “Don’t keep activities” on, start a pick, let MainActivity die in the background, complete the pick in SecondActivity, return — the “Waiting...” message should correctly clear, not stay stuck.

Explain: The activity-result callback itself reliably survives process death. Why can a plain instance variable still get lost even when that’s true?

Task 9: Diagnose a Crash on an Empty Result

Add a second way to leave `SecondActivity` — a “Quick Close” button that finishes the screen with `RESULT_OK` but without attaching any data. Using it crashes `MainActivity` with a null-pointer exception, even though picking a value normally still works fine.

Your job: Find the line in `MainActivity`’s callback that assumes the returned data is never null, and make it null-safe with a sensible fallback instead of crashing.

Verify: Tap “Quick Close” in `SecondActivity` — `MainActivity` should show a reasonable fallback message, not crash.

Explain: In Kotlin, what’s the difference between the safe-call operator (`?.`) and the not-null assertion operator (`!!`) — and why was using the latter here specifically risky?

Wrap-Up

For each task you completed, note which lifecycle callback or communication mechanism was the key to the fix:

Task	Callback / mechanism used
1. Data that never arrives	
2. Activity that refuses to open	
3. Result always reported cancelled	
4. Returned value disappears on rotation	
5. Launcher crashes on tap	
6. False success on cancel	
7. Stack of duplicate activities	
8. Stuck waiting indicator	
9. Crash on empty result	